

OpsFlow 核心流程详解

FlowEngine/PipelineBuilder/PluginService/信号/Celery/超时/回滚/Webhook 全流程源码级解构

2026-06-03

Generated: 2026-06-03 | Source: backend/opsflow/doc/

严格基于 [backend/opsflow/core/]、[signals/]、[tasks.py]、[consumers.py] 全部源代码生成 最后更新于: 2026-06-03

一、完整执行流程全景

前端 DesignCanvas → 保存模板 → 发布版本 → 创建执行(冻结快照) → 启动(异步派发)
→ Celery Worker 执行 → bamboo-engine DAG 遍历 → PluginService 路由 → BasePlugin 执行
→ post_set_state 信号 → 信号处理器(状态/轨迹/日志/超时/WS推送) → 执行完成/Webhook

OpsFlow 的执行引擎基于 `**bamboo-engine 3.x**` (通过 `[BambooDjangoRuntime]` + `[pipeline_api]` 驱动), 所有节点执行和状态追踪通过 `**Celery 异步任务**` + `**Django Signal**` 完成。

二、FlowEngine 引擎 (flow_engine.py)

[FlowEngine] 是核心执行引擎, 封装了所有与 bamboo-engine 的交互。

2.1 类结构

```
class FlowEngine:
    def __init__(self, execution):
        self.execution = execution
        self.template = execution.template
```

2.2 外部 API 方法

方法	触发	说明
`start(sync=False)`	用户点“执行”	设置 status=running → Celery 异步派发 `execute_pipeline_task`
`pause()`	用户点“暂停”	`api.pause_pipeline(runtime, pipeline_id)` → status=paused
`resume()`	用户点“恢复”	`api.resume_pipeline(runtime, pipeline_id)` → status=running
`retry(node_id)`	用户点“重试节点”	`api.retry_node(runtime, node_id)`
`retry_subprocess(node_id)`	用户点“重试子流程”	`api.retry_subprocess(runtime, node_id)`
`skip(node_id)`	用户点“跳过节点”	`api.skip_node(runtime, node_id)`
`force_fail(node_id, ex_data)`	用户点“强制失败”	`api.forced_fail_activity(runtime, node_id)`
`cancel()`	用户点“取消”	`api.revoke_pipeline(runtime, pipeline_id)` → status=cancelled

2.3 run() 核心流程 — 执行 Pipeline

[FlowEngine.run()] 是 Pipeline 执行的入口，严格按以下顺序执行：

```
Step 1 读取 template_snapshot（冻结快照，实现执行隔离）
|
Step 2 validate_pipeline() 安全校验
| 白名单/重试上限/高危回滚路径/备份检查/子流程有效性
|
Step 3 build_bamboo_pipeline() 编译
| 前端 {nodes, edges} 格式 → bamboo-engine 标准 Pipeline Tree
| 包含：节点映射/网关配对/变量注入/超时配置/自动重试策略
|
Step 4 执行前创建 AutoRetryStrategy + NodeTimeoutConfig
|
Step 5 持久化 bamboo_pipeline_id / pipeline / node_id_map 到 context
|
Step 6 _cleanup_pipeline_data() 清理重试残留（删除旧Data/State/ExecutionData记录）
|
Step 7 api.run_pipeline(runtime=BambooDjangoRuntime(), pipeline=pipeline)
| → 异步调度到 Celery er_execute/er_schedule 队列
|
Step 8 (异步) bamboo-engine 内部执行 DAG 遍历
| → 每次节点状态变更触发 post_set_state 信号
| → signals.py 处理器追踪状态
```

失败处理：

- [run_pipeline
通知

- 异常抛出 → `status=failed` → `[rollback_failed_nodes()]` → WS 通知
- 空 pipeline 检测 → 直接标记 failed

三、Pipeline Builder (pipeline_builder/__init__.py)

将前端 `[{nodes, edges}]` 格式转换为 bamboo-engine 可执行格式。

3.1 转换流程

```
输入: {nodes: [{id, label, atom_type, node_type, params, ...}],
      edges: [{from, to, label}]}
```

Step 1 过滤排除节点 (执行方案 `excluded_nodes`)

Step 2 过滤视觉节点 (`start_event/end_event`)

Step 3 扫描边条件 → `edge_conditions + auto_vars`

Step 4 构建邻接表 (`out_edges / in_edges`)

Step 5 创建所有 bamboo-engine 元素
`ServiceActivity / ExclusiveGateway / ParallelGateway / ConvergeGateway / SubProcess`

Step 6 创建 `EmptyStartEvent + EmptyEndEvent`

Step 7 拓扑排序 → BFS 连接节点

- ├── 多根节点 → 自动插入 `ParallelGateway` 合并
- ├── 单出边 → 直接连接
- ├── 多出边 (success/failure) → 自动插入 `ExclusiveGateway` + 条件
- └── 多出边 (其他) → 自动插入 `ParallelGateway`

Step 8 并行网关 → 汇聚网关配对 (BFS 搜索)

Step 9 构建 Data (注入 `target_hosts/global_vars/_execution_id` + 自动变量)

Step 10 `build_tree()` → 返回标准 Pipeline Tree dict

Step 11 节点超时配置 injection

Step 12 ID 映射表 (bamboo UUID → 前端原始节点 ID)

3.2 节点类型映射

node_type	bamboo-engine 元素	说明
start_event	EmptyStartEvent	视觉起点, 不生成有效节点
end_event	EmptyEndEvent	视觉终点
(空)/atom	ServiceActivity	<code>`component_code=opsflow_plugin`</code>
exclusive_gateway	ExclusiveGateway	条件分支
parallel_gateway	ParallelGateway	并行扇出
conditional_parallel_gateway	ConditionalParallelGateway	条件并行
converge_gateway	ConvergeGateway	并行汇聚
subprocess	SubProcess	子流程节点
approval	ServiceActivity + pending_approval	审批节点

四、PluginService 插件执行 (plugin_service_adapter.py)

所有原子操作通过唯一 `[OpsflowPluginComponent]` → `[PluginService]` 路由。

4.1 execute() 执行流程

```

Step 1 从 data.inputs 提取 _atom_type、_plugin_version、_max_retries
|
Step 2 如果 atom_type == 'subprocess_independent'
|   ↳ _execute_independent_subprocess() 独立子流程调度
|
Step 3 从 PLUGIN_REGISTRY 查找插件类 get_plugin(atom_type)
|
Step 4 变量解析: resolve_params(params, resolve_ctx, var_types)
|   resolve_ctx = global_vars + target_hosts + 项目环境变量
|
Step 5 plugin_cls().execute(**params) → 执行真正的业务逻辑
|
Step 6 写回 outputs:
|   _result = success/false
|   stdout / stderr / executor_output
|   _error (失败时)
|
Step 7 _promote_result() 立即将 _result 提升到 pipeline 上下文
|   供 ExclusiveGateway 条件评估使用
|
Step 8 返回 success (True/False)

```

4.2 schedule() 异步轮询

仅 [_need_schedule=True] 的异步插件启用（如 Tower 作业）：

```

pipeline-engine 定期调用 schedule():
→ plugin_cls().schedule(context, **params)
→ 返回 None → 继续等待 (pipeline-engine 下次调度)
→ 返回 True → 执行完成
→ 返回 False → 执行失败

```

4.3 rollback() 回滚

在 Pipeline 失败时由 [rollback_failed_nodes()] 调用：

```

→ plugin_cls().rollback(context, **params)
→ 返回 success/false

```

4.4 独立子流程调度

[_execute_independent_subprocess()] 执行独立子流程调度：

1. 从 inputs 提取 target_template_id / variable_mapping / output_mapping
2. 从 parent_data 获取 execution_id
3. 创建 SubprocessDispatcher
4. dispatcher.start_subprocess() → 创建子 FlowExecution + 变量映射解析
5. 返回 child_execution_id

五、信号系统 (signals/ 包)

bamboo-engine 每次节点状态变更发送 [post_set_state] 信号，由 [on_post_set_state] 接收。

5.1 信号拓扑

```
post_set_state 信号 → on_post_set_state()
|
|— node_id == root_id (Pipeline 级别)
|   ↳ _handle_root_state_change()
|       |— FINISHED → status=completed → WS通知 → Webhook(completed)
|       |— FAILED → status=failed → rollback_failed_nodes → WS通知 → Webhook(failed)
|       |— CANCELLED → status=cancelled → WS通知
|       |— PAUSED → status=paused
|       |— RUNNING → status=running
|
|— node_id != root_id (子节点)
|   |
|   |— FAILED 时检查自动重试:
|       ↳ dispatch_auto_retry() → 有策略且未达上限 → Celery 重试任务 → return
|   |
|   |— _update_execution_node_status() → 执行节点状态
|   |— _update_state_tree() → state_tree JSON 字段
|   |— _record_node_trace() → NodeExecutionTrace 记录
|   |— _update_node_timeout() → Redis 超时追踪
|   |
|   |— RUNNING → 设置 current_node → WS 推送 “running”
|   |
|   ↳ FINISHED/FAILED:
|       |— _log_node_result() → OpsLog 审计日志
|       |— _write_node_trace_log() → 独立日志文件
|       ↳ _notify_node_status() → WS 推送 “completed”/“failed”
```

5.2 信号处理模块

模块	文件	职责
`handlers.py`	信号分发入口	路由根节点/子节点状态、自动重试拦截
`state.py`	节点状态管理	MySQL JSON_SET 原子更新 node_status、state_tree
`trace.py`	轨迹+日志记录	创建 NodeExecutionTrace、OpsLog、日志文件写入
`notify.py`	WS 通知	Celery 任务异步推送节点状态到 WebSocket
`timeout.py`	超时追踪	Redis `opsflow:executing_nodes` 有序集合管理
`helpers.py`	工具函数	重试次数推断、错误提取、审批节点检测

六、Celery 任务系统（tasks.py）

6.1 任务列表

任务	队列	说明
`execute_pipeline_task`	er_execute	主执行入口, max_retries=3
`notify_node_status`	er_execute	WS 节点状态推送（同步 Redis pub/sub）
`notify_execution_completed`	er_execute	WS 执行完成通知（Channels API）
`auto_retry_node_task`	er_execute	自动重试失败节点
`execute_node_timeout_strategy`	er_execute	节点超时策略执行

任务	队列	说明
`webhook_send`	er_execute	Webhook 回调发送
`retry_schedule_execution`	er_execute	调度计划重试执行

6.2 WebSocket 通知机制

支持两种路径：

****Path A — Channels API (execution_completed)：****

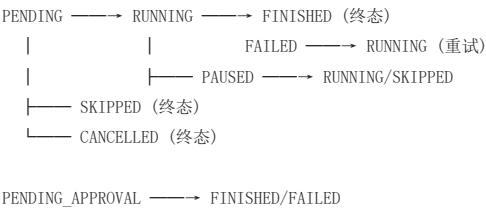
```
notify_execution_completed.delay()
→ channel_layer.group_send("execution_{id}", {"type": "execution.completed"})
→ FlowMonitorConsumer.execution_completed() → send_json
```

****Path B — 同步 Redis pub/sub (node_status)：****

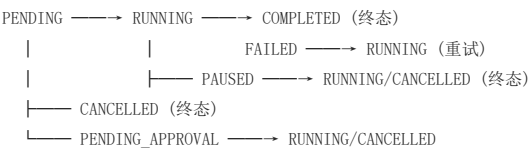
```
notify_node_status.delay()
→ _ws_notify() → 同步 Redis 直连
→ redis.zrange("asgi:g:execution_{id}") 获取 channel names
→ redis.publish("asgi:{channel}", payload)
→ FlowMonitorConsumer.node_status() → send_json
```

七、状态机 (core/states.py)

7.1 NodeState (节点级别)



7.2 PipelineState (执行级别)



八、WebSocket 监控 (consumers.py)

FlowMonitorConsumer

继承 [AsyncJsonWebsocketConsumer]，WebSocket 路径：[ws://host/ws/opsflow/execution/{id}/]

****消息协议：****

消息类型	方向	说明
------	----	----

消息类型	方向	说明
`init_state`	Server→Client	连接时发送当前执行状态快照 (id/status/node_status/current_node)
`node_status`	Server→Client	节点状态变更 (node_id/status/message)
`tower_job_update`	Server→Client	Tower 作业实时状态 (tower_status/progress/artifac
`execution_completed`	Server→Client	执行完成通知 (status)
`ping`	Client→Server	心跳
`pong`	Server→Client	心跳响应

九、自动重试机制 (core/auto_retry.py)

9.1 策略创建

[FlowEngine.run()] 执行前调用 [AutoRetryStrategyCreator.batch_create_strategy()]:

```
扫描 pipeline_tree 中所有节点
→ 读取节点的 max_retries 配置
→ 创建 AutoRetryStrategy 记录 (execution_id/node_id/max_retry_times/interval)
→ bulk_create 批量写入
```

9.2 重试派发

[signals/handlers.py] 拦截到 FAILED 状态后:

```
dispatch_auto_retry(execution, node_id):
→ 查询 AutoRetryStrategy
→ retry_times < max_retry_times?
→ 是: auto_retry_node_task.delay(execution_id, node_id)
    → strategy.retry_times += 1
    → FlowEngine.retry(node_id) → api.retry_node()
→ 否: 不处理, 由正常失败流程处理
```

十、节点超时机制 (core/node_timeout_strategy.py)

10.1 策略创建

[FlowEngine.run()] 中调用 [batch_create_timeout_configs()]:

```
扫描 pipeline_tree 中所有节点的 timeout_seconds
→ 创建 NodeTimeoutConfig 记录
→ 可选策略: forced_fail / forced_fail_and_skip
```

10.2 运行中节点追踪

```
节点进入 RUNNING 状态 → signals/timeout.py:
→ Redis ZADD "opsflow:executing_nodes"
   score = 当前时间 + timeout_seconds
   value = "{execution_id}:{node_id}:{action}"

节点进入终态 → signals/timeout.py:
→ Redis ZREM "opsflow:executing_nodes" 移除
```

10.3 定时扫描

[NodeTimeoutStrategy.dispatch_timeout_nodes()] 由 APScheduler 每 10 秒扫描:

```
Redis ZRANGEBYSCORE "opsflow:executing_nodes" -inf {当前时间}
→ 返回到期节点
→ 每个节点调用 execute_node_timeout_strategy.delay()
→ 执行对应的 Handler: ForcedFailStrategy / ForcedFailAndSkipStrategy
```

十一、安全校验 (core/safety_guard.py)

Pipeline 执行前调用 [validate_pipeline()]:

校验项	说明
白名单校验	检查所有原子是否在 PLUGIN_REGISTRY 中, 未知原子拒绝执行
最大重试上限	单节点重试次数不超过 10
高危回滚路径	标记为 risk_level=high 的节点必须有对应的 rollback 实现
备份检查	高危操作前必须有备份节点
子流程有效性	引用的子流程模板必须存在且已发布
ConflicChecker	检查语义冲突规则 (timeout+auto_retry 冲突、max_retries+skippable 冲突等)

十二、调度计划执行 (core/scheduler_service.py)

[OpsflowScheduler] (APScheduler [BackgroundScheduler]):

```
add_plan(): 添加一次性/CRON 触发器
update_plan(): 更新触发器
remove_plan(): 移除触发器
pause_plan(): 暂停
resume_plan(): 恢复

执行回调 _execute_plan(plan_id):
1. 从 template_snapshot 读取冻结的模板信息
2. 创建 FlowExecution (关联 schedule_plan)
3. FlowEngine.start() 异步执行
4. 更新 last_run_at/total_run_count/next_run_at
```

独立进程启动命令: [python manage.py start_opsflow_scheduler]

十三、变量解析 (core/variable_resolver.py)

13.1 解析引擎

```
resolve_params(params, resolve_ctx, var_types):
    |—— plain 类型 → 直接替换
    |—— splice 类型 → ${key} 模板替换
    |—— split 类型 → 按分隔符拆分
    |—— lazy 类型 → LazyVariable 惰性解析
```

13.2 执行上下文构建

[build_execution_context()] 按优先级合并变量源:

1. project_env (项目环境变量, 最低优先级)
2. global_vars (模板全局变量)
3. hook_variables (提升变量)
4. node_outputs (上游节点输出)
5. system_vars (系统内置变量, 最高优先级)

十四、回滚机制

14.1 Pipeline 级回滚

[FlowEngine.rollback_failed_nodes()]:

1. 遍历 node_status 中 status=failed 的节点
2. 调用 api.get_execution_data_outputs() 获取输出
3. _trigger_plugin_rollback() → 实际调用 BasePlugin.rollback()

14.2 插件级回滚

[PluginService.rollback()]:

1. 从 data.inputs 提取 _atom_type
2. 从 PLUGIN_REGISTRY 查找插件类
3. plugin_cls().rollback(context, **params)
4. 返回 success/false

十五、Webhook 回调 (core/webhook_service.py)

15.1 触发时机

[_handle_root_state_change()] 中:

```
COMPLETED → WebhookService.dispatch(execution, 'completed')
FAILED      → WebhookService.dispatch(execution, 'failed')
```

15.2 发送流程

```
WebhookService.dispatch():  
    → 查询 execution.template.webhooks (所有 enabled 配置)  
    → webhook_send.delay(webhook_id, execution_id, event)  
  
WebhookService.send():  
    → 构建 request_body (含执行状态/节点结果/时间)  
    → HMAC 签名 (如果配置了 secret)  
    → POST 到 callback URL  
    → 失败重试 (retry_count/retry_interval)  
    → 记录 WebhookLog
```

十六、执行数据清理

`[_cleanup_pipeline_data()]` 在重试执行时清理 bamboo-engine 残留数据:

```
PipelineData.objects.filter(node_id__in=all_node_ids).delete()  
PipelineExecutionData.objects.filter(node_id__in=all_node_ids).delete()  
PipelineState.objects.filter(node_id__in=all_node_ids).delete()  
PipelineSchedule.objects.filter(node_id__in=all_node_ids).delete()  
PipelineNode.objects.filter(node_id__in=all_node_ids).delete()  
PipelineExecutionHistory.objects.filter(node_id__in=all_node_ids).delete()  
PipelineCallbackData.objects.filter(node_id__in=all_node_ids).delete()
```

十七、完整时序图

